Legal Document QA System

Course Name:Generative AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Jogendar Das Bairagi	EN22ME304045
2.	Roshan Bankar	EN22EL301072
3.	Vivek Gangrade	EN22CS3011098
4.	Maahi Maheshwari	EN22ME304057
5.	Shubhangi Prajapat	EN22ME304094

Group Name : 04D10

Project Number : DO-17

Industry Mentor Name : Mr. Yash Mittal

University Mentor Name : Prof. Divya Kumawat

Academic Year : 2025-26

Table of content

1.Problem Statement & Objectives

1. Problem Statement
2. Project Objectives
3. Scope of the Project

2.Proposed Solution

1. Key features
2. Overall Architecture / Workflow
3. Tools & Technologies Used

3.Results & Output

1. Screenshots / outputs
2. Reports / dashboards / models
3. Key outcomes

4.Conclusion

5.Future Scope & Enhancements

6.Reference

1. Problem Statement & Objectives

1.1 Problem Statement

Legal professionals, researchers, and students routinely spend considerable time manually reviewing dense, multi-page legal contracts and agreements to extract specific clauses, definitions, or obligations. This manual process is time-consuming, error-prone, and inaccessible to non-experts. There is a pressing need for an intelligent system that can understand natural language queries and retrieve precise, document-grounded answers from complex legal PDFs.

1.2 Project Objectives

- Develop an AI-powered chatbot capable of answering natural language questions based on uploaded legal documents.
- Implement a Retrieval-Augmented Generation (RAG) pipeline to ground AI responses in actual document content, preventing hallucinations.
- Enable semantic search over legal PDFs using vector embeddings and FAISS for fast, accurate context retrieval.
- Integrate the Groq LPU-accelerated Llama 3.1 model to deliver sub-2-second response times.
- Build a user-friendly Gradio interface accessible via a web browser without any installation requirements for end users.
- Ensure data privacy by adopting a volatile (in-memory) storage model that leaves no persistent data footprint after a session.

1.3 Scope of the Project

This project covers the end-to-end design and implementation of a Legal Document QA System. The scope includes:

- Multiple PDF ingestion and text extraction using PyPDFLoader.
- Text chunking, embedding generation, and semantic indexing via FAISS.
- A conversational chat interface built with Gradio.
- Integration with the Groq Cloud API for LLM inference.

2. Proposed Solution

2.1 Key Features

- Natural Language Q&A: Users can ask questions in plain English and receive factual answers drawn directly from the uploaded contract.
- Retrieval-Augmented Generation (RAG): Ensures every answer is grounded in the actual document, drastically reducing AI hallucination risk.
- Semantic Search with FAISS: Finds the most relevant paragraphs using vector similarity (L2 distance) rather than simple keyword matching.
- High-Speed Inference via Groq LPU: Delivers responses in under 2 seconds, making the experience feel conversational.
- Chunk Overlap Strategy: 100-character overlap between 600-character chunks ensures legal definitions spanning chunk boundaries remain intact.
- Privacy-First Design: All document data is stored in-memory and automatically purged when the session ends — nothing is written to disk.
- Zero-Setup Web Interface: The Gradio UI is accessible via any web browser at localhost:7860 with no client-side installation.

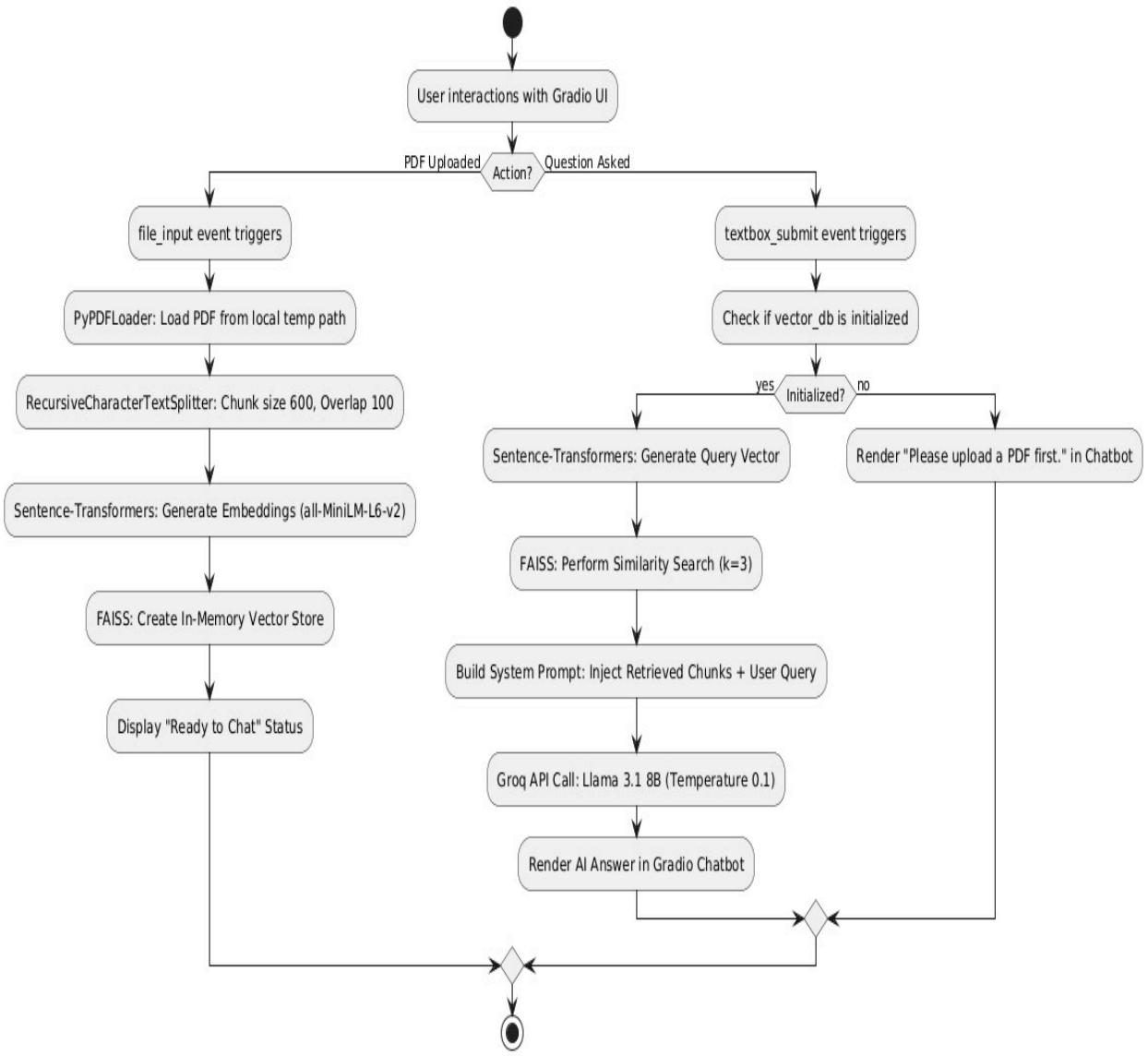
2.2 Overall Architecture / Workflow

The system follows a Three-Tier Architecture:

- Presentation Tier (Gradio): A web-based frontend for PDF uploads and conversational chat. Maintains session state within the browser.
- Intelligence Tier (Lang Chain + Groq) : Orchestrates the RAG pipeline — embedding, retrieval, and LLM inference. The Llama 3.1 model acts as the "Legal Expert."
-

Knowledge Base Tier (FAISS): An in-memory vector database that stores document chunks as mathematical embeddings, enabling millisecond-speed semantic.

System Design Diagram:

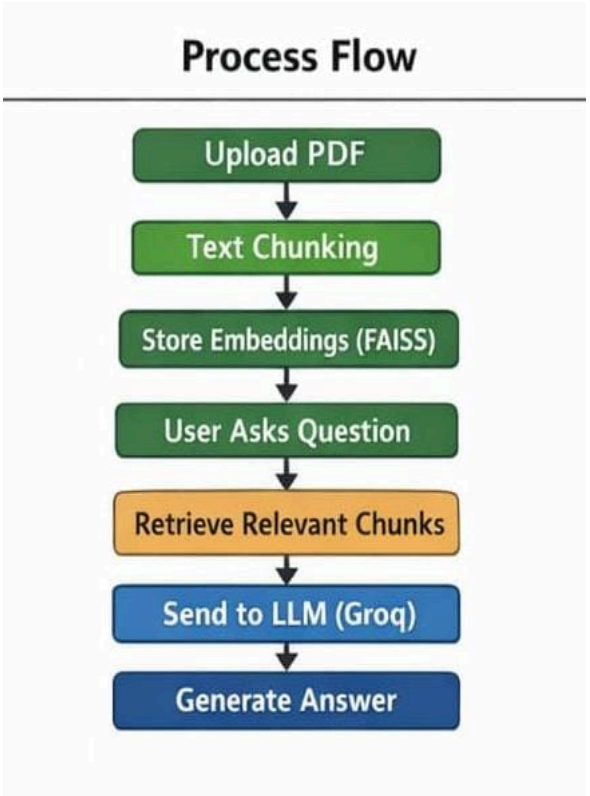


Process Flow:

The user journey follows three key stages:

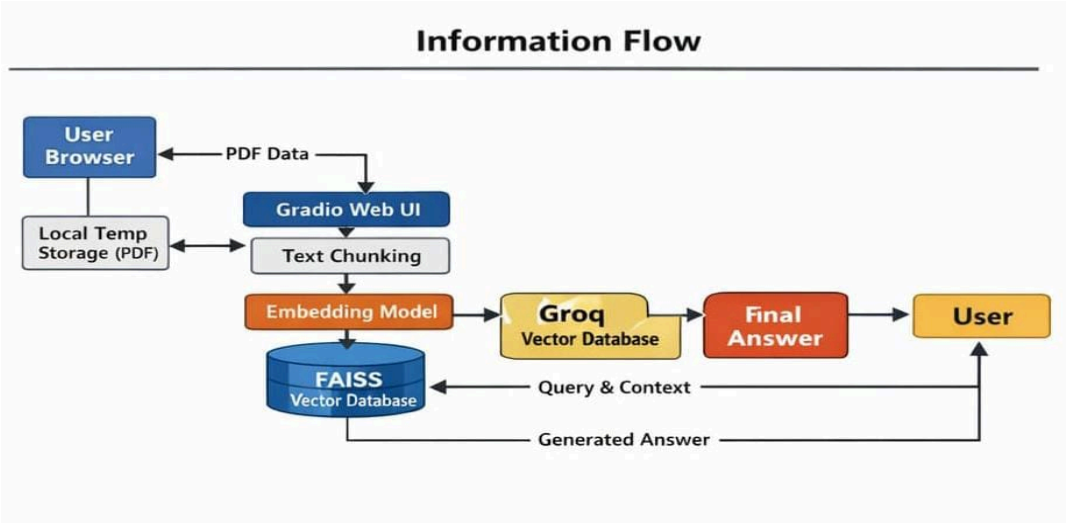
- Upload: User uploads a legal PDF through the Gradio interface.
- Index: The system reads, chunks, embeds, and stores the document text into the FAISS in-memory vector database.

Chat: The user asks a question; the system retrieves the top-3 relevant chunks and sends them to the Groq API to generate a grounded answer.



Information Flow:

- PDF Data: User Browser → Gradio Local Temp Folder.
- Vector Data: Embedding Model → In-Memory FAISS Index.
- Contextual Prompt: Local Python Environment → External Groq API.



Components Design:

Component	Technology	Responsibility
Document Loader	Py PDF Loader	Parsing PDF structure and extracting raw text.
Text Splitter	Recursive Character Text Splitter	Breaking text into 600-character chunks with 100-character overlap.
Embeddings	Hugging FaceEmbeddings	Generating semantic signatures for text chunks.
Vector Store	FAISS	Performing K-Nearest Neighbor (KNN) search for relevant context.
LLM Inference	Groq	Generating natural language answers based on retrieved context.

2.3 Tools & Technologies Used

- **Python** : Backend development and orchestration.
- **Lang Chain** : RAG pipeline management; connects embeddings, vector store, and LLM.
- **FAISS (Facebook AI Similarity Search)** : High-speed in-memory vector database for semantic search.
- **Hugging Face Embeddings** : Converts text chunks into dense vector representations.
- **Groq Cloud API** : Hosts the Llama 3.1 model with LPU-accelerated inference.
- **Gradio** : Rapid web UI framework for the chatbot interface.
- **Py PDF Loader** : Parses and extracts text from PDF documents.
- **Recursive Character Text Splitter** : Splits extracted text into 600-char chunks with 100-char overlap.

3. Results & Output

3.1 Screenshots / Outputs

The following screenshots demonstrate the working Legal Document QA System interface Figure 1: Gradio Interface — Multiple PDF Upload & Chat Panel

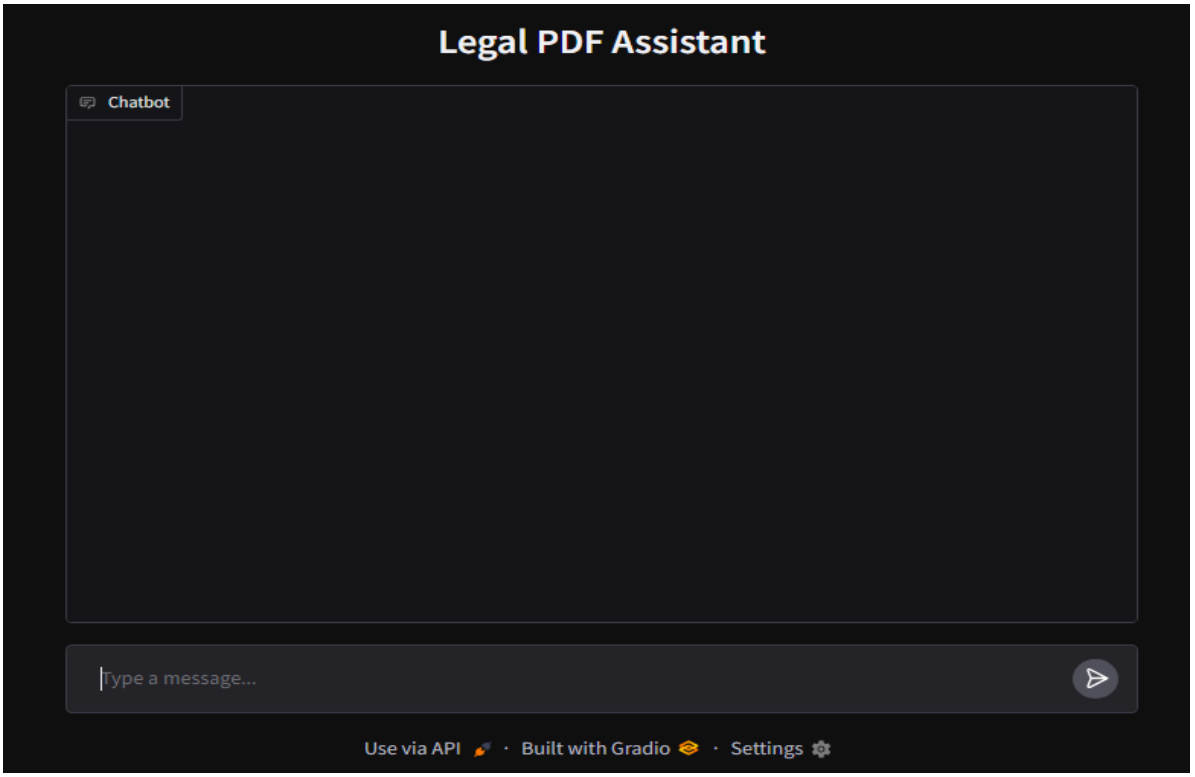


Figure 2: Sample Query — Extracting Clause Information from Contract

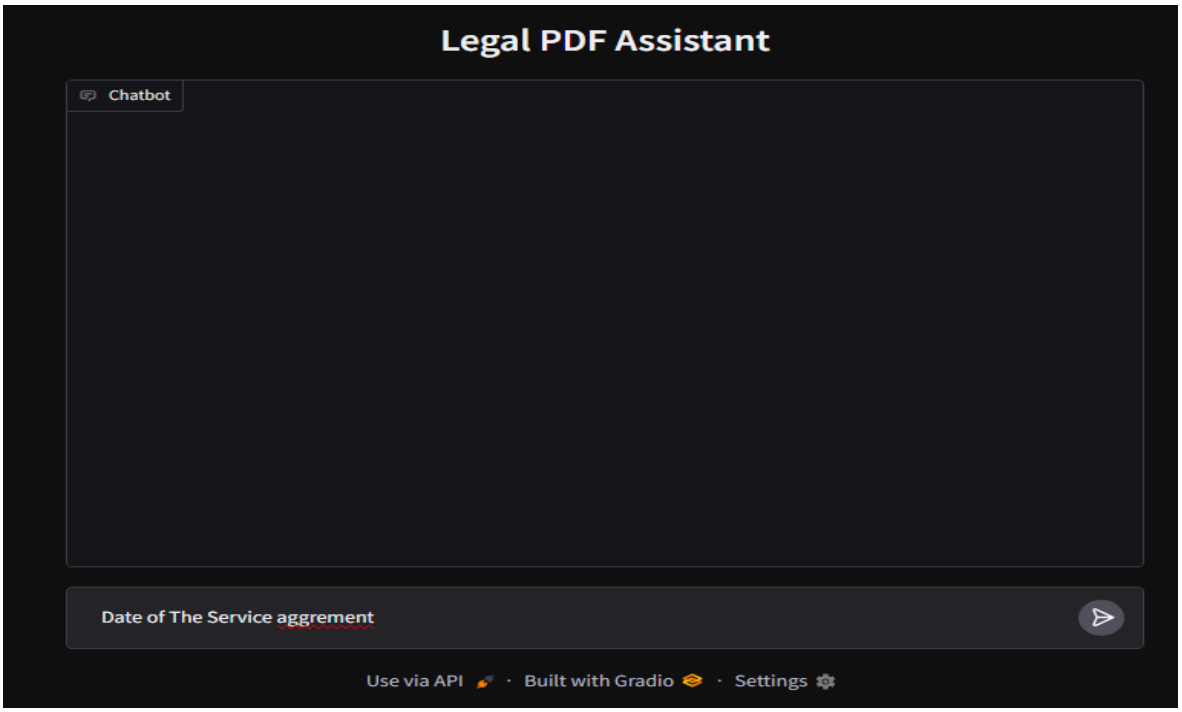
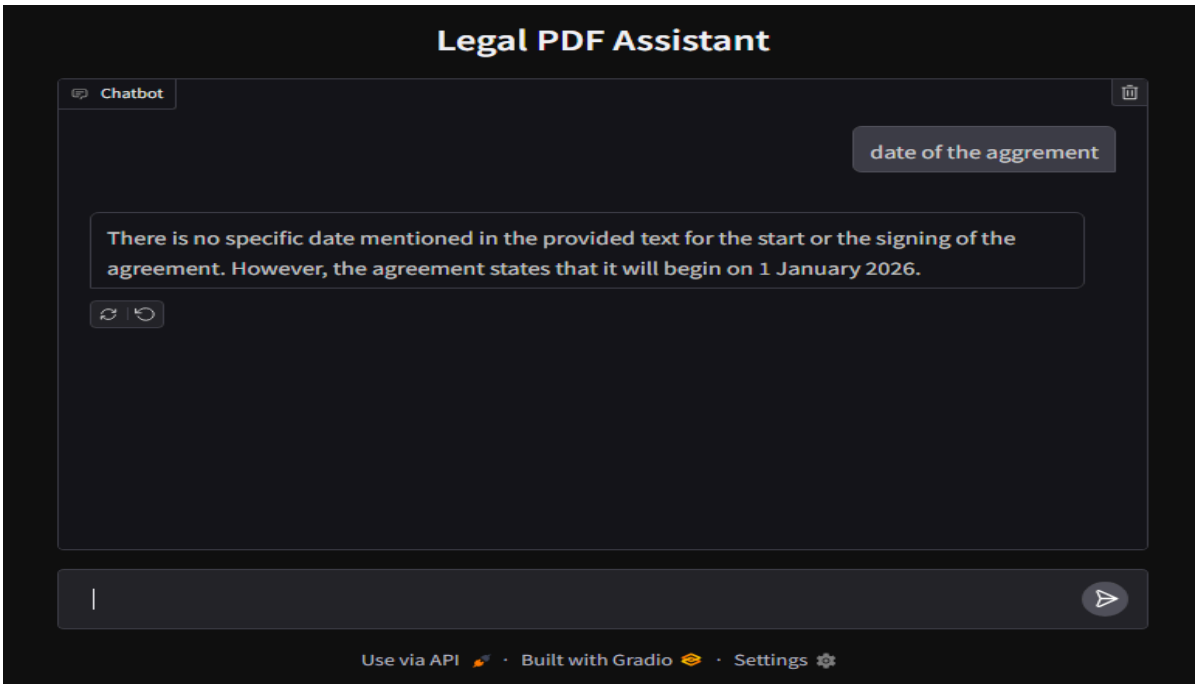
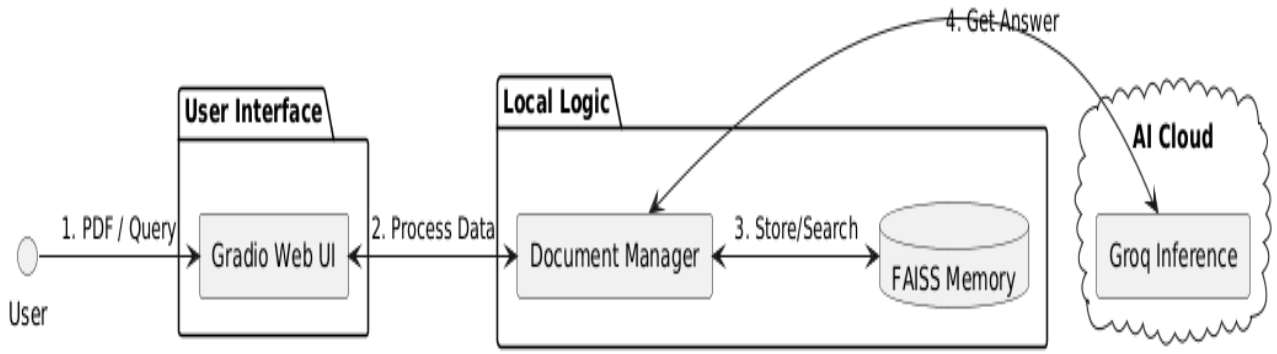


Figure 3: Multi-Turn Conversation with Document-Grounded Answers



3.2 Data Model & Pipeline Output

The diagram below shows the data transformation pipeline from raw PDF input to indexed vector chunks:



3.3 Key Outcomes

- Successfully built a fully functional Legal Document QA chatbot using RAG architecture.
- Achieved sub-2-second response times for document-grounded queries using Groq LPU inference.
- Demonstrated accurate retrieval: the system correctly identifies and surfaces relevant clauses from multi-page legal PDFs.
- Zero data persistence validated — all document data is cleared upon session termination.
- Chunk overlap strategy confirmed effective for legal documents where clause definitions may span paragraph boundaries.

System successfully prevents hallucination by restricting the LLM to answer only from retrieved context

4. Conclusion

The Legal Document QA System successfully demonstrates how Generative AI and Retrieval-Augmented Generation (RAG) can be applied to solve a real-world problem in the legal domain. By combining semantic search (FAISS), high-speed LLM inference (Groq + Llama 3.1), and an accessible web interface (Gradio), the project delivers an intelligent assistant capable of extracting precise, document-grounded answers from complex legal contracts.

A key learning from this project was the critical role of RAG in making AI responses reliable and trustworthy. Unlike general-purpose chatbots, this system's answers are always anchored to the specific document provided, significantly reducing the risk of legal misinformation. The volatility-first data model further underscores the importance of privacy-by-design when handling sensitive documents.

The project also deepened the team's understanding of vector embeddings, LangChain orchestration pipelines, chunking strategies for legal text, and the trade-offs between local and cloud-based LLM inference. Overall, it provided hands-on experience bridging the gap between theoretical Generative AI concepts and production-aware application development.

5. Future Scope & Enhancements

The current system provides a strong proof-of-concept foundation. Several enhancements can extend its capabilities towards a production-ready legal AI platform:

- Persistent Storage & User Authentication: Integrate a persistent vector database (e.g., ChromaDB with disk storage or Pinecone) and an authentication layer so users can save and revisit past document sessions.
- Advanced Security: Implement end-to-end encryption for document transfers, role-based access control, and audit logging for enterprise deployments.

- Fine-Tuned Legal LLM: Train or fine-tune a domain-specific model on legal corpora to improve accuracy on specialized legal terminology, jurisdictional nuances, and contract structures.

- Citation & Source Highlighting: Enhance the interface to visually highlight the exact paragraph or clause in the original PDF from which each answer was derived.
- Multilingual Support: Extend document processing and QA capabilities to support legal documents in regional languages, broadening accessibility.

6 .References

1. Groq LPU Architecture Whitepaper. Groq Inc.
2. Lang Chain RAG Documentation. https://docs.langchain.com/
3. FAISS: A Library for Efficient Similarity Search. Facebook AI Research.
4. Hugging Face Embeddings Documentation. https://huggingface.co/docs/
5. Gradio: Build & Share Delightful Machine Learning Apps. https://gradio.app/